

UI 设计文档 · v1.0

会议语音纪要

实时 ASR · 实时纠错 · 每 5 分钟 AI 阶段整理
自定义系统提示词 · 原始转写与纠错文本双输出

Tauri 桌面应用

明 / 暗主题

Apple 设计原则

安装包本体约 20 MB

+ 新建会议

实时记录

进行中

历史会议

28

提示词模板

4

今天

后端工程师 · 二面

进行中 · 42 分钟

后端工程师 · 一面

10:00 · 已整理 · 平均分 3.8

产品评审周会

昨天 · 已整理

语音识别

本地 · 流式

AI 整理

云端模型

本机识别占用

CPU 11%

实时转写

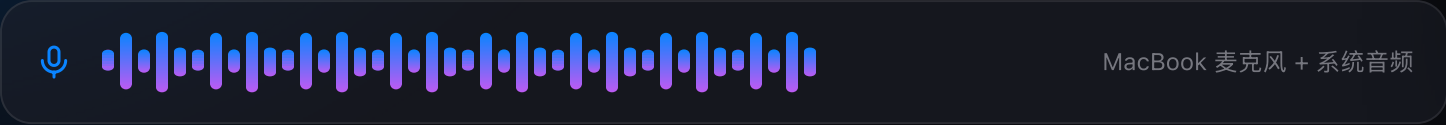
实时纠错 已开启

热词 · Kafka / Rust / K8s +12

纠错后

对照

原始



- 面

面试官 00:39:52

那你们线上的消息队列是怎么选型的？当时为什么没有用 RabbitMQ？
- 候

候选人 00:40:07 已纠错 3 处

我们用的是 ~~卡夫卡~~ **Kafka**，主要是吞吐量的考虑，日均消息大概两亿条。服务是用 ~~锐斯~~ **特 Rust** 重写的，部署在 ~~K8S~~ **K8s** 上，延迟从 120 毫秒降到了 30 毫秒。
- 面

面试官 00:41:26

重写的时候怎么保证数据一致性？有没有做灰度？
- 候

候选人 00:41:48 识别中

我们是双写加对账的方式，先灰度百分之五的流量，然后逐步放开，中间|

暂停

标记要点

立即整理

已识别 6,420 字 · 已纠错 47 处

3:12

下一次阶段整理

每 5 分钟自动 · 基于面试复盘模板

5 分钟

阶段纪要

关键点

候选人评分

第 8 段 00:35 – 00:40

刚整理

系统设计：

主导消息平台从 RabbitMQ 迁移至 Kafka，日均 2 亿条。

性能：

Rust 重写后延迟 120ms → 30ms，有量化数据。

不足：

对一致性方案描述偏笼统，未提对账细节。

第 7 段 00:30 – 00:35

项目背景：

电商履约系统，团队 8 人，负责核心链路。

亮点：

主动推动监控告警体系，故障恢复时间缩短一半。

待追问 一致性 & 灰度回滚

是否演练过回滚？对账不一致时的处理流程？

+ 新建会议

实时记录

进行中

历史会议

28

提示词模板

4

今天

后端工程师 · 二面

进行中 · 42 分钟

后端工程师 · 一面

10:00 · 已整理 · 平均分 3.8

产品评审周会

昨天 · 已整理

语音识别

本地 · 流式

AI 整理

云端模型

本机识别占用

CPU 11%

实时转写

实时纠错 已开启

热词 · Kafka / Rust / K8s +12

纠错后

对照

原始

MacBook 麦克风 + 系统音频

- 面试官

00:39:52

那你们线上的消息队列是怎么选型的？当时为什么没有用 RabbitMQ？
- 候选人

00:40:07

已纠错 3 处

我们用的是 ~~卡夫卡~~ **Kafka**，主要是吞吐量的考虑，日均消息大概两亿条。服务是用 ~~锐斯~~ **特 Rust** 重写的，部署在 ~~K8S~~ **K8s** 上，延迟从 120 毫秒降到了 30 毫秒。
- 面试官

00:41:26

重写的时候怎么保证数据一致性？有没有做灰度？
- 候选人

00:41:48

识别中

我们是双写加对账的方式，先灰度百分之五的流量，然后逐步放开，中间|

暂停

尸 标记要点

✧ 立即整理

已识别 6,420 字 · 已纠错 47 处

3:12

下一次阶段整理

每 5 分钟自动 · 基于面试复盘模板

5 分钟

阶段纪要

关键点

候选人评分

第 8 段

00:35 – 00:40

刚整理

系统设计：

主导消息平台从 RabbitMQ 迁移至 Kafka，日均 2 亿条。

性能：

Rust 重写后延迟 120ms → 30ms，有量化数据。

不足：

对一致性方案描述偏笼统，未提对账细节。

第 7 段

00:30 – 00:35

项目背景：

电商履约系统，团队 8 人，负责核心链路。

亮点：

主动推动监控告警体系，故障恢复时间缩短一半。

待追问

一致性与灰度回滚

是否演练过回滚？对账不一致时的处理流程？

我的模板

面试复盘与评分

每 5 分钟 · 含条件筛选

项目周会纪要

每 5 分钟 · 待办与结论

客户访谈

每 10 分钟 · 需求与异议

自定义

空白模板

+ 新建模板

面试复盘与评分

系统提示词决定 AI 如何整理每个 5 分钟阶段，以及会议结束后的最终输出。

主题任务

你是一名资深技术面试记录员。会议为「后端工程师」岗位面试，请持续记录候选人的项目经历、技术细节、量化成果与回答中的不足，并标注面试官尚未追问的点，便于面试后复盘。

最终输出要求

1. 整体总结（不超过 200 字）

2. 按维度打分（1–5）：技术深度 / 系统设计 / 沟通 / 稳定性，并附证据原话

3. 指定条件筛选结果，逐条给出「符合 / 不符合 / 未提及」

4. 附：原始转写与纠错后全文

筛选条件（会议结束时逐条判定）

条件	判定方式	权重
3 年以上后端经验	候选人自述年限 ≥ 3	必须
有 Rust 或 Go 生产经验	提到并描述过线上项目	加分
能量化项目成果	至少一处具体数字	加分

变量 {会议主题} {阶段起止} {热词表}

已输入 386 字 · 自动保存于 1 秒前

输出预览

阶段纪要

最终报告

第 3 段 · 00:10–00:15（示例）

- 项目经历：负责订单履约链路，团队 8 人。
- 技术细节：Kafka 分区按订单号哈希，幂等消费。
- 不足：未说明如何处理重复消息的兜底。

筛选进度

- 3 年以上后端经验符合
- Rust 或 Go 生产经验符合
- 能量化项目成果未提及

阶段整理间隔5 分钟

整理模型云端 · 可换

密钥存储系统钥匙串

阶段整理携带前文摘要

每段带上上一段结论，保持上下文连贯

我的模板

面试复盘与评分

每 5 分钟 · 含条件筛选

项目周会纪要

每 5 分钟 · 待办与结论

客户访谈

每 10 分钟 · 需求与异议

自定义

空白模板

+ 新建模板

面试复盘与评分

系统提示词决定 AI 如何整理每个 5 分钟阶段，以及会议结束后的最终输出。

主题任务

你是一名资深技术面试记录员。会议为「后端工程师」岗位面试，请持续记录候选人的项目经历、技术细节、量化成果与回答中的不足，并标注面试官尚未追问的点，便于面试后复盘。

最终输出要求

1. 整体总结（不超过 200 字）

2. 按维度打分（1–5）：技术深度 / 系统设计 / 沟通 / 稳定性，并附证据原话

3. 指定条件筛选结果，逐条给出「符合 / 不符合 / 未提及」

4. 附：原始转写与纠错后全文

筛选条件（会议结束时逐条判定）

条件	判定方式	权重
3 年以上后端经验	候选人自述年限 ≥ 3	必须
有 Rust 或 Go 生产经验	提到并描述过线上项目	加分
能量化项目成果	至少一处具体数字	加分

变量 {会议主题}

{阶段起止}

{热词表}

已输入 386 字 · 自动保存于 1 秒前

输出预览

阶段纪要

最终报告

第 3 段 · 00:10–00:15（示例）

项目经历：负责订单履约链路，团队 8 人。

技术细节：Kafka 分区按订单号哈希，幂等消费。

不足：未说明如何处理重复消息的兜底。

筛选进度

3 年以上后端经验

符合

Rust 或 Go 生产经验

符合

能量化项目成果

未提及

阶段整理间隔

5 分钟

整理模型

云端 · 可换

密钥存储

系统钥匙串

阶段整理携带前文摘要

每段带上上一段结论，保持上下文连贯



总结 纠错后转写 原始转写

总结由 12 个阶段纪要合并生成

整体总结

4.1/5

综合评分 · 建议进入三面

候选人 4 年后端经验，主导消息平台由 RabbitMQ 迁移至 Kafka（日均 2 亿条），并用 Rust 重写核心服务，延迟由 120ms 降至 30ms，成果可量化。一致性方案（双写加对账、5% 灰度）描述清晰，但未演练回滚；沟通流畅，对监控告警体系有主动推动。

维度	评分	证据
技术深度	4.5	"分区按订单号哈希，消费端做幂等键去重"
系统设计	4.0	"先灰度百分之五，再逐步放开"
沟通表达	4.0	结构清晰，能主动补充背景
稳定性意识	3.8	提到监控告警，但缺少回滚演练

转写对照 · 原始 vs 纠错

共 47 处修改

ASR 原始 · 00:40:07

我们用的是卡夫卡，主要是吞吐量的考虑。服务是用锐斯特重写的，部署在 K8S 上。

纠错后

我们用的是 **Kafka**，主要是吞吐量的考虑。服务是用 **Rust** 重写的，部署在 **K8s** 上。

条件筛选

3 年以上后端经验

符合 · 4 年

Rust 或 Go 生产经验

符合 · Rust

能量化项目成果

符合 · 3 处

结论

全部满足

导出内容

整体总结与评分



纠错后全文（含说话人）



ASR 原始转写



纠错对照（标注 47 处修改）



Markdown

PDF

TXT

SRT

导出为 Markdown

全部数据保存在本机，可选择仅本地保存，不上传音频。

总结

纠错后转写

原始转写

总结由 12 个阶段纪要合并生成

整体总结

4.1

/5

综合评分 · 建议进入三面

候选人 4 年后端经验，主导消息平台由 RabbitMQ 迁移至 Kafka（日均 2 亿条），并用 Rust 重写核心服务，延迟由 120ms 降至 30ms，成果可量化。一致性方案（双写加对账、5% 灰度）描述清晰，但未演练回滚；沟通流畅，对监控告警体系有主动推动。

维度	评分	证据
技术深度	4.5	"分区按订单号哈希，消费端做幂等键去重"
系统设计	4.0	"先灰度百分之五，再逐步放开"
沟通表达	4.0	结构清晰，能主动补充背景
稳定性意识	3.8	提到监控告警，但缺少回滚演练

转写对照 · 原始 vs 纠错

共 47 处修改

ASR 原始 · 00:40:07

我们用的是卡夫卡，主要是吞吐量的考虑。服务是用锐斯特重写的，部署在 K8S 上。

纠错后

我们用的是 **Kafka**，主要是吞吐量的考虑。服务是用 **Rust** 重写的，部署在 **K8s** 上。

条件筛选

3 年以上后端经验

符合 · 4 年

Rust 或 Go 生产经验

符合 · Rust

能量化项目成果

符合 · 3 处

结论

全部满足

导出内容

整体总结与评分

纠错后全文（含说话人）

ASR 原始转写

纠错对照（标注 47 处修改）

Markdown

PDF

TXT

SRT

⌵ 导出为 Markdown

全部数据保存在本机，可选择仅本地保存，不上传音频。

实现方案与设计规范

Tauri 2 + 流式本地 ASR · 安装包本体约 20 MB · 模型按需下载



技术选型（为小体积）

层	选择	体积
桌面壳	Tauri 2（系统 WebView，Rust 后端）	约 5–8 MB
界面	Svelte + 纯 CSS 令牌，无大型 UI 库	< 1 MB
流式 ASR	sherpa-onnx（Zipformer 中英 int8）	运行库约 12 MB
语音模型	首次启动按需下载，可选云端 ASR	约 70–130 MB，可不装
纠错 / 整理	热词规则本地；LLM 走 API，可换本地模型	0
存储	SQLite，全部在本机	—

对比 Electron 同等功能安装包通常 100 MB 以上；上表体积为估算，以实际打包为准。

设计令牌 · 明 / 暗

暗色

底 #0A0B0F · 文字 #F5F5F7

浅色

底 #EEF0F6 · 文字 #1D1D1F

Apple 设计原则落地

- 材质：侧栏与卡片用 blur 20–28px 半透明，越大越厚
- 动效：弹簧 阻尼 1.0 / 响应 0.35s；下拉面板才用 0.8 回弹
- 实时：转写行逐字追加，纠错处 0.3s 淡入高亮，不打断阅读
- 可访问：跟随系统明暗，遵循减弱动效 / 透明度
- 字距：大标题 −0.02em，正文 0，符合尺寸相关字距